X01｜一张工单怎样从文字变成可以读回的事实？

抓住三次关键交接：**输入变成 API 可用值；第一笔事务把本地意图保存；合作方确认再交给第二笔事务。** 最后一次 GET 读到的是已经提交的状态，不是客户端仍握着的同一个对象。

从文字到数据库，哪一步才留下工单？

当前问题	关系或短用法	记到什么程度
输入 "3" 后，何时能发请求？	`strip()` 清理边缘空白，`int()` 转整数，再查 1–3 范围；`urgent` 在转换失败，`-3` 在范围失败，都不进入 HTTP 调用。	把 A 册的值流接到 B 册请求入口；先分清“没发出”和“发出后被拒绝”。
客户端 dict 怎样交给 API？	HTTPX 的 `json=` 编成 JSON 请求体；接收方从字节解析自己的对象。	**两端数据表示相通，内存对象不共享。** 编码本身不证明服务接受。
客户端检查过，API 为何还校验？	任意调用者都可绕过该客户端；X 的 `TicketCreate` 要求 priority 严格整数 1–3，拒绝额外字段等。	本例 `"priority":"3"` 返回 422；不能照搬其他示例允许转换的规则。
`validation_passed` **表示工单已保存吗？**	只说明输入成为 API 可用对象；随后还需核心数据条件检查与创建事务。	**合法输入与持久事实分开。**
第一笔事务一起保存哪些事实？	tickets 行含工单、create_key 与输入摘要；ticket_events 保存创建事件；编号计数同时前进，最后 COMMIT。	认清一个事务的共同边界；本例没有单独的幂等记录表。
事件 INSERT 失败，工单与编号怎样变化？	本例事务失败时一起撤回：无新票、无新事件，下一编号仍是 42。	票、依据与编号要共同成立；失败前打印日志不能替数据库提交。

`BEGIN IMMEDIATE` 在 SQLite 事务开始时申请写事务；完整函数与 SQL 查原章，不在表里拼写入配方。回看：「同一个 3，怎样离开客户端进程？」「客户端已经检查过，API 为什么还要检查？」「第一笔事务为什么同时保存工单、意图和事件？」。

本地已保存，为什么还要经过外部确认和第二笔事务?

当前问题	关系或短用法	记到什么程度
T-42 为什么同时 open 和 pending?	open 表示工单仍待处理；assignment_status=pending 表示派单确认尚未在本地收束。	**工单状态与派单状态直接分清。**有票不等于已有安排。
工单创建 key 与 `assignment:T-42` 标识同一动作吗?	前者识别本地创建意图；后者识别合作方派单意图。request ID 则关联本次处理。	稳定意图、外部意图、本次请求分开，不能互换字段。
派单请求把全部工单交出去吗?	本例 body 只含 ticket_id、device_id；另带 Idempotency-Key 和 X-Request-ID header。	按交接用途认字段范围；不发送 subject/priority 不等于没带关联头。
等待合作方时，还持有创建事务吗?	第一笔事务已提交，之后才发网络请求；本地没有持有写事务等回复。	能解释为什么后来 timeout 不能跨网络撤销对方效果。
合作方返回 PA-1，本地就 confirmed 了吗?	adapter 先核 HTTP、JSON 和字段；`finalize_assignment()` 再用第二笔事务保存确认与 assignment_confirmed 事件。	**收到可用外部结果，不等于本地已持久化。**
partner_confirmed 为什么不等于工单关闭?	派单确认后工单仍 open；有人安排处理不代表故障修好。顶层确认标记从已保存的派单状态推导。	公开结果要依据已提交状态，不依据打算或刚收到的候选回复。
GET T-42 会重新派单吗?	它查询本地行并返回公开工单，不调用合作方或模型。	认清只读交接；创建 response 与后续 GET 要能对上。

三个"状态"可同时成立：HTTP `201` 是本次创建结果，工单 `open` 是故障处理状态，`partner_confirmed` 是派单子流程状态。数据库物理列叫 `status`，API 映射为 `ticket.ticket_status`；改公开名称没有改变存储所有者。

回看：「open 与 pending 为什么会同时成立?」「合作方确认以后，为什么还要第二笔事务?」。

政策请求交回的是什么，为什么不能替工单作决定?

当前问题	关系或短用法	记到什么程度
创建以后，政策草稿是自动附带的下一步吗?	处理人员另发 `POST /tickets/T-42/policy-draft`，用新的 request ID；应用读工单和获准资料，再请求 Fake Model。	两项任务分别发起；草稿分支不是创建事务的一部分。
“用户报告离线”和“政策要求确认离线”一样吗?	前者是工单报告，后者是行动前条件；两者都不证明设备已检测或当前配置已备份。	**描述、要求与已经完成的事实分开。**
`accepted_draft` **允许直接执行恢复吗?**	应用检查结构、引用和本例窄政策条件后，交回可供人核对的草稿；不写成工单完成事实。	候选被接受的用途是草稿，不自动获得动作权限。
没有资料、引用不属于本次资料，各停在哪里?	无适用资料则 abstained，模型调用为零；引用越界则 needs_human。	先判依据是否存在，再判候选能否采用；HTTP 200 不保证候选可用。
模型建议 `close_ticket`**，谁决定能否调用?**	本应用只有 `ticket:read` 权限，返回 needs_human / tool_permission_denied；tool calls/effects 都为 0，工单仍 open。	**模型建议与应用授权分开。** 拒绝后还需看调用和状态证据。
Fake 检查通过，能说明真实模型可靠吗?	本例检索是封闭资料筛选，claim 检查是固定文字条件，Fake 提供受控候选。	只能验证本例检查位置与规则；真实资料和模型效果回 E 册另做评测。

回看: 「另一个请求为什么只产生草稿?」。完整关系 trace 入口是 `walkthrough.py success`，见「看见同一程序里的这些交接」: 它新建临时 SQLite，TestClient 进入真实 API，HTTPX MockTransport 接住外部边界，结束后清理临时数据。其创建 201、重放 200 和计数是受控实验预期，尚未经过真实监听端口。